

MODULE-III

ARRAYS

3.1 Arrays

- ✓ An Array is a special and powerful data structure and it is used to store, process and print large amounts of data.
- ✓ **“An array is a collection of similar type of items (elements) stored sequentially (continuously) one after the other in memory”.**

Ex: 1. `int A[5];     // Array of 5 Integers`

A[0]	10
A[1]	20
A[2]	30
A[3]	40
A[4]	50

- ✓ The Elements in the Array A can be accessed using the common name A, but with different index.
- ✓ The Element '10' is called 0th Element and it can be accessed using the Subscript 0 (called Index '0') along with name of the array 'A'.

2. `char B[5];                     //Array of 5 Characters`

B[0]	'A'
B[1]	'r'
B[2]	'r'
B[3]	'a'
B[4]	'y'

3. `float C[5];                     //Array of 5 floats`

C[0]	12.56
C[1]	234.20
C[2]	215.60
C[3]	322.50
C[4]	123.45

- ✓ **An 'Index' is also called as Subscript ([]). It is used to indicate the position of an element in the Array.**

Basic Properties of the Arrays

1. All the elements in an array should be of the same data type.
2. The elements are stored continuously in the memory. (For example, in the array `char B[5]`, if the first address is 1000 then the data is stored contiguously in the addresses 1000, 1001, 1002 and so on).
3. The Subscript (index) of first item is always zero.
4. Each element of an array is accessed using the name of the Array, but with different subscript.

5. The Index of the Array is always an Integer number can't be float.

Ex: a [1] or a [5].

a [1.5] is an Error.

3.2 Classification of Arrays

1. Single Dimensional Array
2. Multi-Dimensional Array

3.2.1 Single Dimensional Array

Definition:

Single dimensional array (One-dimensional array) is a linear list consisting of related data items of same data type and in the memory, all the data items are stored contiguously in memory locations one after the other.

Or

An array with one index is called as Single dimensional array (One-dimensional array)

Declaration of Single dimensional arrays

- ✓ As we declare the variables before they are used in a program, an array must also be declared and defined before it is used using following syntax.
- ✓ **Syntax**

```
data_type array_name[size];
```

Where,

data_type: data type can be int, float, char etc.

array_name: name of the array which is a valid C variable.

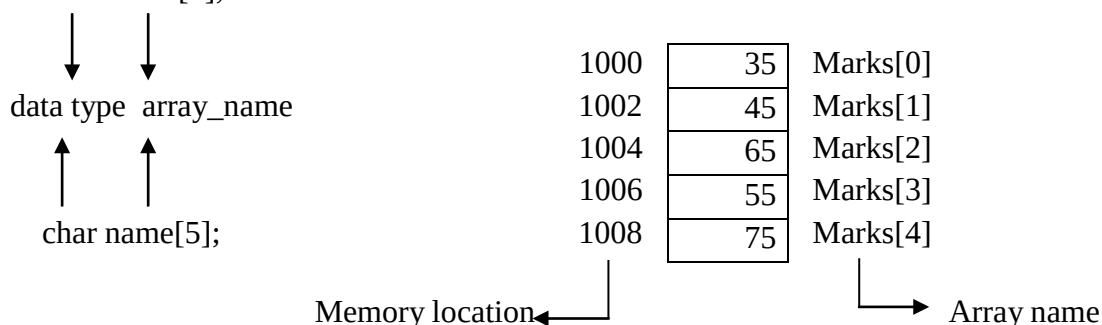
size: it is the number of elements in the array.

Complete declaration ends with Semicolon.

Ex: int Marks[5];

Declares Marks as an array consisting of 5 elements of integer data type.

Ex: int Marks[5];



- ✓ 5 memory locations are reserved. sizeof(int) is 2 bytes, 2*5=10 bytes are reserved.
- ✓ 5 memory locations are reserved. sizeof(char) is 1 bytes 1*5=5 bytes are reserved.

Initialization of Single dimensional arrays

- ✓ Once an array is declared it must be initialized with some values using Initialization.

“Process of assigning the values to the individual elements of an array is called as Initialization of an array”.

Syntax

```
data_type array_name[size]={v1,v2,----,vn};
```

data_type: it can be int, float, char etc.

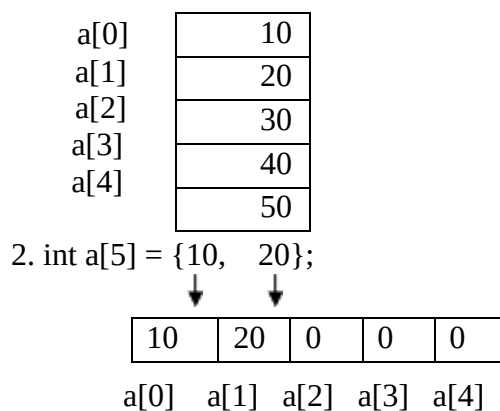
array_name: it is the name of the array.

size: it is the number of elements in the array.

v₁,v₂, ---- ,v_n are the values and should be enclosed within ‘{’ and ‘}’ separated by commas.

Ex: 1. `int a[5] = {10, 20, 30, 40, 50};`

- ✓ The compiler allocates 5 memory locations and these locations are initialized with the integer values in the order specified.



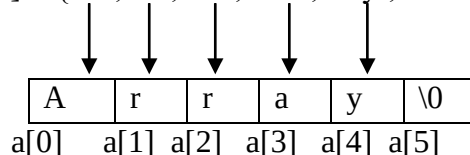
- ✓ When the numbers of initial values are lesser than declared array size then those many elements will be initialized and other memory locations are automatically initialized to 0's, in case of numeric arrays.
- ✓ It is ‘partial array initialization’.

3. `int a[ ] = {10, 20, 30, 40, 50};`

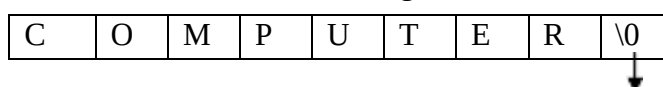
↓
Size not specified

- ✓ If we have not specified the size then the compiler will calculate the size of the array based on the number of initial values.

4. `char a[6] = {'A', 'r', 'r', 'a', 'y', '\0'};`



5. `char b[ ] = "COMPUTER"; //String Initialization`



↓
Null character at the end of the string

Reading/Writing Single dimensional arrays

- ✓ We can easily read and write the array elements using for loop.
- ✓ Reading the elements of an array from the keyboard can be done using **for loop** and **“scanf()” function**.
- ✓ Writing/Displaying the elements of an array to the screen can be done using **for loop** and **“printf()” function**.
- ✓ C code or C instruction to read ‘n’ elements of an array is shown below:

```
for(i=0;i<n;i++)
{
    scanf("%d",&a[i]);
}
```

- ✓ C code or C instruction to display/write ‘n’ elements of an array is shown below:

```
for(i=0;i<n;i++)
{
    printf("%d\n",a[i]);
}
```

Example C programs of Single dimensional array**1. Write a C Program to read n items from keyboard and display them on the monitor.**

```
#include<stdio.h>
```

```
void main( )
```

```
{
    int n,a[10],i;
    printf("Enter the size of array:");
    scanf("%d",&n);
    printf("Enter the array elements:\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    printf("\nEntered elements are:\n");
    for(i=0;i<n;i++)
    {
        printf("%d\n",a[i]);
    }
}
```

Output:

```
Enter the size of array:
5
Enter the array elements:
1 2 3 4 5
Entered elements are:
1
2
3
4
5
```

2. Write a C program to find the sum and average of n array elements.

```
#include<stdio.h>
```

```
void main( )
```

```
{
    int n,a[10],i,sum=0;
    float avg=0;
    printf("Enter the number of elements in the array:");
    scanf("%d",&n);
    printf("Enter the array elements:\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
}
```

Output:

```
Enter the size of array:
3
Enter the array elements:
12
34
50
Sum= 96
Average =32
```

```

    for(i=0;i<n;i++)
    {
        sum=sum+a[i];
    }
    avg=sum/n;
    printf("Sum=%d\n Average=%f",sum,avg);
}

```

3. Write a C program to find the largest of 'n' numbers in the array.

```

#include<stdio.h>
void main( )
{
    int n,a[20],i,large=-1;
    printf("Enter the size of array:\n");
    scanf("%d",&n);
    printf("Enter the array elements:\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    for(i=0;i<n;i++)
    {
        if(a[i]>large)
        {
            large=a[i];
        }
    }
    printf("The largest number in the array is=%d\n",large);
}

```

Output:

```

Enter the size of array:
5
Enter the array elements:
10
20
30
40
50
The largest number in the
array is= 50

```

4. Write a C program to print the smallest of 'n' numbers in the array.

```

#include<stdio.h>
void main( )
{
    int n,a[20],i,small=9999;
    printf("Enter the number of elements in the array:\n");
    scanf("%d",&n);
    printf("Enter the array elements:\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    for(i=0;i<n;i++)
        if(a[i]<small)
        {
            small=a[i];
        }
    printf("The smallest number in the array is=%d",small);
}

```

Output:

```

Enter the size of array:
5
Enter the array elements:
10
20
30
40
50
The largest number in the
array is= 10

```

3.2.2 Two dimensional arrays (Multi-dimensional arrays)

- ✓ Arrays which are specified with 2 subscripts (2 set of square brackets [][]) are called 2-dimensional arrays.
- ✓ Arrays with two or more dimensions are called **Multi-dimensional arrays**.
- ✓ In 2-Dimensional Array, the first index indicates the 'row size' (the number of rows) and second index indicates the 'column size' (the number of columns).

Ex: `int a[10][10];` //Two-dimensional array
`int b[3][4][5];` //Three-dimensional array

Declaration of Two-dimensional arrays

- ✓ As we declare the variables before they are used in a program, an array must also be declared before it is used using the following syntax.

Syntax:

```
data_type array_name [row_size][col_size];
```

data_type: It can be int, float, char etc.

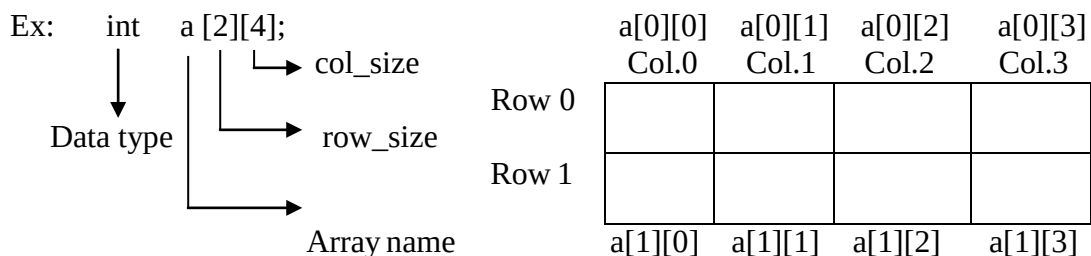
array_name: It is the name of the array.

row_size: It is the number of rows in the array.

col_size: It is the number of columns in the array.

Semicolon is must at the end.

- ✓ The size used during declaration of the array is useful to reserve the specified memory locations.



- ✓ The array 'a' is a 2-dimensional array with 2 rows and 4 columns. This declaration informs the compiler to reserve 8 locations ($2 \times 4 = 8$ locations, $2 \times 8 = 16$ bytes in total) continuously one after the other.

Initialization of 2-dimensional arrays

- ✓ As we initialize a variable to the required value, we can initialize the individual elements of the array during initialization.

Syntax

```
data_type array_name[row_size][col_size]={
    {a1,a2, ----,an},
    {b1,b2, ----,bn},
    .....
    {z1,z2,---- ,zn}
};
```

data_type: It can be int, float, char etc.

array_name: It is the name of the array.

row_size: It is the number of rows in the array.

col_size: It is the number of columns in the array.

a₁ to a_n are the **values** assigned to **1st row**, **b₁ to b_n** are the **values** assigned to **2nd row** and so on.

Ex: 1. `int a[4][3]= {{11,22,33}, {44,55,66},{77,88,99},{10,20,30} };`
 The array has 4 rows and 3 columns.

		Columns →		
		0	1	2
Rows ↓	0	11	22	33
	1	44	55	66
	2	77	88	99
	3	10	20	30

2. `int a[4][3]= {`

`{11,22},`
`{33,4},`
`{55,66},`
`{77,88}`

`};`

The array has 4 rows and 3 columns.

		Columns →		
		0	1	2
Rows ↓	0	11	22	0
	1	33	44	0
	2	55	66	0
	3	77	88	0

//This is a partial array initialization

Reading/Writing Two dimensional arrays

- ✓ To read the 2-dimensional array, we have to use two for loops along with `scanf()`, where the outer for loop indicates the number of rows to be read and the inner for loop indicates the number of columns to be read.
- ✓ To read the 'n' array elements of Two dimensional array:

```
for(i=0;i<m;i++)
{
    for(j=0;j<n;j++)
    {
        scanf("%d",&a[i][j]);
    }
}
```

- ✓ To write/print the 2-dimensional array elements, we have to use two for loops along with `printf()`, where the outer for loop indicates the number of rows to be printed and the inner for loop indicates the number of columns to be printed.
- ✓ To print the 'n' array elements of Two dimensional array:

```
for(i=0;i<m;i++)
{
    for(j=0;j<n;j++)
    {
        printf("%d\t",a[i][j]);
    }
    printf("\n");
}
```

Example Programs:**1. Write a C Program to read and print 2-dimensional array.**

```
#include<stdio.h>
```

```
void main( )
```

```
{
```

```
    int a[10][10],i,j,m,n;
```

```
    printf("Enter the number of Rows and Columns of the Array:\n");
```

```
    scanf("%d%d",&m,&n);
```

```
    printf("Enter the Array elements:\n");
```

```
    for(i=0;i<m;i++)
```

```
    {
```

```
        for(j=0;j<n;j++)
```

```
        {
```

```
            scanf("%d",&a[i][j]);
```

```
        }
```

```
    }
```

```
    printf("Entered array elements are:\n");
```

```
    for(i=0;i<m;i++)
```

```
    {
```

```
        for(j=0;j<n;j++)
```

```
        {
```

```
            printf("%d\t",a[i][j]);
```

```
        }
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
}
```

Output:

Enter the number of Rows and
Columns of the Array:

2 2

Enter the Array elements:

1

2

3

4

Entered array elements are:

1 2

3 4

2. Write a C program to add 2 matrices A and B and store sum in C.

```
#include<stdio.h>
void main( )
{
    int a[10][10],b[10][10],c[10][10],i,j,m,n;
    printf("Enter the size of matrix:\n");
    scanf("%d%d",&m,&n);
    printf("Enter the elements of Matrix A:\n");
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&a[i][j]);
        }
    }
    printf("Enter the elements of Matrix B:\n");
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&b[i][j]);
        }
    }
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            c[i][j]=a[i][j]+b[i][j];
        }
    }
    printf("The Resultant Matrix C is:\n");
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            printf("%d\t",c[i][j]);
        }
        printf("\n");
    }
}
```

Output:

Enter the size of matrix:

2 2

Enter the elements of Matrix A:

1 2

3 4

Enter the elements of Matrix B:

5 6

7 8

The Resultant Matrix C is:

6 8

10 12

3. Write a C program to find the largest of 'n' array elements in a 2-Dimensional array.

```
#include<stdio.h>
void main( )
{
    int a[10][10],i,j,m,n,big=-1;
    printf("Enter the size of matrix:\n");
    scanf("%d%d",&m,&n);
    printf("Enter the elements of Matrix A:\n");
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&a[i][j]);
        }
    }
}
```

```

    }
}
for(i=0;i<m;i++)
{
    for(j=0;j<n;j++)
    {
        if(a[i][j]>big)
        {
            big=a[i][j];
        }
    }
}
printf("The largest element is:%d\n",big);
}

```

Output:

Enter the size of matrix:

2 2

Enter the elements of Matrix A:

10 20

30 40

The largest element is: 40

4. Write a C program to find the smallest of 'n' array elements in a 2-Dimensional array.

#include<stdio.h>

void main()

```

{
    int a[10][10],i,j,m,n,small=9999;
    printf("Enter the size of matrix:\n");
    scanf("%d%d",&m,&n);
    printf("Enter the elements of Matrix A:\n");
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&a[i][j]);
        }
    }
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            if(a[i][j]<small)
            {
                small=a[i][j];
            }
        }
    }
    printf("The smallest element is:%d\n",small);
}

```

Output:

Enter the size of matrix:

2 2

Enter the elements of Matrix A:

10 20

30 40

The largest element is: 10

